

Bilinear Rank as Proof Cost: Fast Matrix-Multiplication Schemes and Field-Specific Flip Search for zkML

Greg Sidebottom

Research note — logic repo (<https://github.com/gsidebottom/logic>), matmul track, 2026-07-11. Companion to the 3×3 additive-complexity paper (`doc/matmul_adds_paper.md`, artifacts DOI 10.5281/zenodo.21240904), the rank-48 rigidity paper, and the flower note. The engine of §5 is `src/bin/flip23p.rs` in this repository.

Abstract

Zero-knowledge machine learning (zkML) — proving that a neural-network inference was computed correctly without re-executing or revealing it — is bottlenecked by the prover, whose work in circuit-based proof systems is counted in **multiplication constraints**: additions and constant-coefficient linear combinations are free. This is the exact inversion of the hardware cost model that keeps fast matrix multiplication out of GPUs, and it makes **bilinear rank** — the number of multiplications in a Strassen-like scheme — the direct cost driver for matrix products between witness values (private weights; attention). Three consequences. (1) Known schemes already pay: recursive rank-48 4×4 blocking proves a 768×768 witness $\times$ witness product with $\approx 4\times$ fewer constraints than the naive circuit, penalty-free because the schemes’ additions cost nothing. (2) Coefficient obstructions vanish: schemes over $\mathbb{Q}$ with dyadic (or any rational) coefficients are exact over a prime field, and numerical stability is not a concept. (3) Most interestingly, **rank is field-dependent** — rank 47 for 4×4 exists modulo 2 with no characteristic-0 equivalent known — and zkML fixes a short list of concrete fields (Goldilocks, BabyBear, M31, the BN254/BLS12-381 scalar fields) over which nobody has searched. We port our rational flip-graph engine to $\mathbb{F}_p$ (`flip23p`, Goldilocks first): over a prime field the engine is strictly stronger than its $\mathbb{Q}$ parent — no coefficient growth, no magnitude caps, and every coincidence-solving λ is admissible, so the coplanarity structure that rationality left mostly unusable becomes fully spendable. A verified rank-22 3×3 or rank-47 4×4 over a proof field would be a shippable constraint-count reduction with no analog in the characteristic-0 literature. We state the cost model precisely, including the honest carve-outs (sumcheck/GKR proves multiply matrices in $\sim n^2$ prover work and do not care about rank; public-weight linear layers cost no constraints), inventory which of our existing artifacts transfer, and report the port’s first measurements.

1 zkML in one page

A **zero-knowledge proof** (ZKP) lets a prover convince a verifier that a statement holds — here, “this output is what model M computes on input x ” — without the verifier re-executing the computation, and optionally without revealing x or M . Two families dominate. **SNARKs** (succinct non-interactive arguments of knowledge; Groth16 [6], PLONK [5]) give constant-or-polylog proof sizes over elliptic-curve-pairing fields such as the BN254 and BLS12-381 scalar fields (~ 254 -bit primes). **STARKs** [2] replace pairings with hashes, gain transparency and plausible post-quantum security, and run over small “FFT-friendly” primes chosen for fast number-theoretic transforms — Goldilocks $p = 2^{64} - 2^{32} + 1$ (Plonky2 [12]), BabyBear $p = 2^{31} - 2^{27} + 1$ (RISC Zero [13]), and M31 $= 2^{31} - 1$ (Circle STARKs [8]).

The asymmetry that defines the field: verification is cheap, **proving is 10^3 – $10^6\times$ the native computation**, all of it exact arithmetic in the proof field. zkML applications — proving a proprietary model was faithfully executed (model privacy), proving an inference inside a blockchain rollout, auditable AI where the checker is cheap — inherit that overhead on every multiply of every layer. Making matrix multiplication cheap *inside a proof* is therefore a different optimization problem from making it fast on silicon, with a different — in fact opposite — cost model.

2 What proving charges

The standard arithmetization, **R1CS** (rank-1 constraint systems, as in Groth16 [6]), expresses a computation as constraints of the form

$$\left(\sum_i a_i w_i\right) \times \left(\sum_i b_i w_i\right) = \left(\sum_i c_i w_i\right)$$

over the proof field, where w is the witness vector and the a_i, b_i, c_i are *constants*. Each constraint carries exactly one multiplication of witness values; the linear combinations inside the parentheses — additions, subtractions, scalings by arbitrary field constants — are free: they are folded into the constraint’s constant vectors and cost the prover nothing. PLONK-family systems [5] charge per gate/row with the same shape (custom gates and lookup arguments shift constants around but preserve the headline: **bilinear multiplications of witnesses are the metered resource**).

For an $n \times n$ product $C = A \cdot B$ where both operands are witnesses, each entry $C_{ij} = \sum_k A_{ik} B_{kj}$ is a sum of n witness×witness products; a sum of n products is not one rank-1 quadratic form — it takes n constraints. The naive circuit therefore costs n^3 constraints. Two situations make both operands witnesses in zkML: **private weights** (the commercially central case — the model owner proves inference without revealing the model) and **attention** ($QK^\top$ and scores $\cdot V$ multiply two activation matrices — witness×witness even when all weights are public). Conversely, a linear layer with *public* weights is a constant linear map and costs zero multiplication constraints; rank optimization has nothing to offer there.

A toy example, 2×2 over $\mathbb{F}_{17}$. Take witnesses $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, $B = \begin{pmatrix} 5 & 6 \\ 7 & 0 \end{pmatrix}$; then $C = A \cdot B \equiv \begin{pmatrix} 2 & 6 \\ 9 & 1 \end{pmatrix} \pmod{17}$. The naive circuit spends one constraint per product — eight constraints of the shape $u_1 = A_{11} \times B_{11}$ — and output sums like $C_{11} = u_1 + u_2$ cost nothing further (they are linear). Strassen’s rank-7 scheme spends **seven**: its first product is the single constraint

$$(w_{A_{11}} + w_{A_{22}}) \times (w_{B_{11}} + w_{B_{22}}) = w_{P_1},$$

where all four operand additions sit *inside* the constraint’s linear forms, free. There is exactly **one witness per proof**: a single vector w over $\mathbb{F}_{17}$ holding every wire value — a conventional leading 1 (which lets constraints use constants), the inputs, the seven products, the outputs. Concretely, all 20 coordinates:

$$w = (1 \mid \underbrace{1, 2, 3, 4}_A \mid \underbrace{5, 6, 7, 0}_B \mid \underbrace{8, 1, 6, 8, 0, 5, 3}_{P_1 \dots P_7} \mid \underbrace{2, 6, 9, 1}_C).$$

The subscripted symbols above are coordinates of this one vector, and each constraint is checked by reading them off: for P_1 , $(1 + 4) \cdot (5 + 0) = 25 \equiv 8 \checkmark$, and likewise $P_2 = 7 \cdot 5 = 1$, $P_3 = 1 \cdot 6 = 6$, $P_4 = 4 \cdot 2 = 8$, $P_5 = 3 \cdot 0 = 0$, $P_6 = 2 \cdot 11 = 5$, $P_7 = (-2) \cdot 7 = 3$ (all mod 17). The free output combinations reproduce C exactly: $C_{11} = P_1 + P_4 - P_5 + P_7 = 19 \equiv 2$, $C_{12} = P_3 + P_5 = 6$, $C_{21} = P_2 + P_4 = 9$, $C_{22} = P_1 - P_2 + P_3 + P_6 = 18 \equiv 1$. (The prover commits to w and proves the constraints hold; the verifier never sees w — that is the “zero-knowledge”. Note the naive circuit’s witness carries eight product coordinates to Strassen’s seven, so lower rank also shortens the committed vector.) Seven constraints instead of eight — 12.5% at one level, compounding to $n^{2.807}$ under recursion — and dyadic coefficients would be equally at home: $\frac{1}{2}$ is just the constant 9 in $\mathbb{F}_{17}$ ($2 \cdot 9 = 18 \equiv 1$). This is §3’s story in miniature; the rank-23 and rank-48 schemes are the same picture with better ratios.

3 Bilinear rank is constraint count

A bilinear scheme of rank r computes an $m \times m$ block product with r products of linear forms. In R1CS each product is exactly one constraint — the linear forms slide into the constraint’s free linear parts — so an $m \times m$ witness×witness block product costs r constraints instead of m^3 , and recursion compounds it: $T(n) = r T(n/m)$, i.e. $n^{\log_m r}$ constraints total, **with zero cost for the combination additions at every level**. The additions that price fast schemes out of silicon are literally free here; there is no numerical-stability tax because arithmetic is exact; and coefficients may be any field constants — dyadic $\frac{1}{2}$ ’s are simply the constant $(p+1)/2 \pmod{p}$.

Idealized constraint counts for a witness×witness product (recursion to scalar base; real circuits stop at a base tile, which shifts absolute numbers but not the ordering):

scheme (exponent)	$n = 768$	vs naive	$n = 4096$	vs naive
naive (3.000)	4.53×10^8	1.0×	6.87×10^{10}	1.0×
$3 \times 3 : 23$ (2.854)	1.72×10^8	2.6×	2.05×10^{10}	3.4×
Strassen $2 \times 2 : 7$ (2.807)	1.26×10^8	3.6×	1.39×10^{10}	4.9×
$4 \times 4 : 48$ (2.792)	1.14×10^8	4.0×	1.23×10^{10}	5.6×
$3 \times 3 : 22$ (2.814), <i>if found</i>	1.32×10^8	3.4×	—	—
$4 \times 4 : 47$ (2.785), <i>if found</i>	1.09×10^8	4.2×	—	—

Two readings. First, **the wins in the top half are available today**: the rank-48 scheme [3, 10] with dyadic coefficients drops into any R1CS/PLONK matmul gadget as-is (our repository carries the verified scheme and its minimized linear networks). Second, the *marginal* rows show why new records matter here more than in silicon: each rank step is a few percent per recursion level, compounding — and unlike hardware, nothing eats the margin.

4 The honest carve-outs

Rank is **not** the lever everywhere:

- **Sumcheck/GKR provers do matmul in $\sim n^2$ prover field-ops** (Thaler [4]; GKR [7]); zkCNN [9] and successors build dedicated zkML provers on this line, and where such a prover applies, bilinear rank is irrelevant. The rank lever applies to the (large) circuit-based world: general-purpose SNARK toolchains, zkVMs executing matmul as arithmetic, PLONK/R1CS pipelines such as EZKL [11], and folding-based IVC (Nova [14]) whose per-step cost is the step circuit’s constraint count.
- **Public-weight linear layers are free**; rank helps witness×witness products only — private weights and attention.
- **Constraint count is not wall-clock**: prover time also includes commitments (multi-scalar multiplications / hashing); fewer constraints shrink those too, but constants differ by system.
- **Nonlinearities dominate some workloads** (range checks, lookups for ReLU/quantization); the matmul share is large in attention/convolution-heavy models, smaller in lookup-heavy quantized pipelines.

5 Field-specific rank: the open frontier, and flip23p

Bilinear rank depends on the coefficient field. The emblem: AlphaTensor’s rank-47 4×4 scheme exists over $\mathbb{F}_2$ [3], no characteristic-0 rank-47 is known, and our companion work proved the only $\mathbb{Q}$ -usable rank-48 class *rigid* under a large certified move system — evidence that characteristic 0 is genuinely constrained in ways a fixed finite field need not be. zkML nominates a short, concrete list of fields that matter commercially — Goldilocks, BabyBear, M31, BN254-Fr, BLS12-381-Fr — and, to our knowledge, **no rank search has ever been run over any of them**: the flip-graph literature works over $\mathbb{F}_2$ and small extensions [15], the explicit records over $\mathbb{Z}/\mathbb{Q}$. A verified 3×3 rank-22 — or 4×4 rank-47 — over Goldilocks would be a new kind of record with an immediate consumer: 2–4% fewer constraints per recursion level in deployed proof systems, compounding as in §3.

flip23p is our rational flip engine (splits, λ -flips, reductions, canonical gauge, exact Brent verification — the machinery of the companion papers) re-based onto $\mathbb{F}_p$, Goldilocks first. Over a prime field the engine is *strictly stronger* than its $\mathbb{Q}$ parent:

1. **No coefficient growth.** $\mathbb{Q}$ -walks need magnitude caps and dyadic bookkeeping; field elements don’t grow. Every cap in the $\mathbb{Q}$ engine is deleted, not loosened.
2. **Every solved flip is admissible.** Over $\mathbb{Q}$, a coincidence ($f_i + \lambda f_j \propto f_m$, the Cramer-solved move that makes flips productive) is usable only when λ is dyadic — the filter discarded most solutions. Over $\mathbb{F}_p$ every nondegenerate solution is a legal move: the coplanarity structure (“H4’s raw material” in the flower note) becomes fully spendable.

3. **The gauge simplifies:** monic normalization (leading coefficient 1, scalars folded into the c -slot); proportional $\Leftrightarrow$ equal, with none of $\mathbb{Q}$'s content/sign bookkeeping.

First measurements (2026-07-11; §8): the 55-addition record scheme loads and verifies over Goldilocks (729 Brent equations mod p); the census gate matches the $\mathbb{Q}$ engine exactly (9 shared pairs, weight 177, distinct-rows 40). One honest surprise: the seed's solved-move silence is *not* the dyadic filter's fault — even with all field λ available, its 9 shared pairs admit no coincidence solution; the coplanarity geometry genuinely fails there. Mobility arrives at once elsewhere: a 25-second smoke storm from the most mobile census seed executed 1.3M moves with 369K solved flips and ~ 10 K reductions — a far higher solved-flip fraction than the $\mathbb{Q}$ storm ever achieved, the freed λ paying immediately. No sub-23 rank has appeared in smoke tests (nor was one expected at that effort); the campaigns are the point.

6 What transfers from the existing program

- **Engines and discipline.** flip48/flip23's storm, closure, census, novelty and thin-storm machinery ports mechanically (v1 of flip23p carries storm/census/closures; the beam-chase and exhaustive campaign modes follow). The operational rules hard-won in the companion papers — completion logs, full-frontier dumps, control baselines, budget caps — apply verbatim.
- **Seeds.** Every scheme we hold is a valid $\mathbb{F}_p$ scheme: the 17,376 database classes, our 53 lifted classes, the 46 storm-new classes, and the 467 mod-2-invisible dyadic $\mathbb{Q}$ -schemes (denominators are field constants). The mobility census that ranked $\mathbb{Q}$ -seeds re-runs unchanged.
- **Verified gadgets.** The Lean pipeline that certified the 55-add scheme extends naturally: a scheme-to-circuit emitter (circom / Halo2 templates for the 23- and 48-multiplication blocks) with a machine-checked correctness certificate is a deliverable the proof-systems world actually values.
- **What does not transfer:** the additive-complexity results (55 adds, the no-54 theorem). Additions are free here; that program's value stays in classical hardware and in witness-generation speed.

7 Program

(1) Goldilocks campaigns: storm + closure portfolios over the census-ranked seeds; targets, in order of plausibility $\times$ value: any new-class harvest over $\mathbb{F}_p$, rank 22 at 3×3 , rank 47 at 4×4 (the 4×4 port is the same edit at 16 coordinates). (2) Small-field variants: BabyBear and M31 (31-bit arithmetic, $\sim 4\times$ faster walks), then BN254-Fr. (3) Rectangular tiles: transformer shapes want $\langle m, n, k \rangle$ records; the engine generalizes as flip48 $\rightarrow$ flip23 did. (4) Verified gadget emitter + benchmark note: constraint-count measurements in an EZKL-style stack, naive vs recursive-scheme, with Lean-certified gadget templates.

8 Reproduction

```
cargo build --release --bin flip23p
./target/release/flip23p --census --dir matmul/mm23          # gates
./target/release/flip23p --dir matmul/seeds23/13a5bd4n \
  --seconds 25 --threads 4 --out matmul/found23p             # smoke storm
# any verified rank <= 22 over Goldilocks lands in
# found23p/RECORDP_rank*.txt and is announced loudly
```

Acknowledgments

This work was carried out in an extended interactive collaboration with Claude (Anthropic; the Fable 5 and Opus 4.8 models), which implemented the engines and drafted this text under the author's direction and review.

References

- [1] V. Strassen. *Gaussian elimination is not optimal*. Numer. Math. 13:354–356, 1969.
- [2] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev. *Scalable, transparent, and post-quantum secure computational integrity*. ePrint 2018/046.
- [3] A. Fawzi et al. *Discovering faster matrix multiplication algorithms with reinforcement learning*. Nature 610:47–53, 2022.
- [4] J. Thaler. *Time-optimal interactive proofs for circuit evaluation*. CRYPTO 2013.
- [5] A. Gabizon, Z. Williamson, O. Ciobotaru. *PLONK*. ePrint 2019/953.
- [6] J. Groth. *On the size of pairing-based non-interactive arguments*. EUROCRYPT 2016.
- [7] S. Goldwasser, Y. T. Kalai, G. N. Rothblum. *Delegating computation: interactive proofs for muggles*. STOC 2008.
- [8] U. Haböck, D. Levit, S. Papini. *Circle STARKs*. ePrint 2024/278.
- [9] T. Liu, X. Xie, Y. Zhang. *zkCNN*. CCS 2021.
- [10] J.-G. Dumas, C. Pernet, A. Sedoglavic. arXiv:2506.13242; A. Novikov et al. *AlphaEvolve*. arXiv:2506.13131.
- [11] EZKL. github.com/zkonduit/ezkl.
- [12] Polygon Zero. *Plonky2*. Whitepaper, 2022.
- [13] RISC Zero. *zkVM documentation*, 2023.
- [14] A. Kothapalli, S. Setty, I. Tzialla. *Nova*. CRYPTO 2022.
- [15] M. Kauers, J. Moosbauer. *Flip graphs for matrix multiplication*. arXiv:2212.01175.
- [16] G. Grassi et al. *Poseidon*. USENIX Security 2021.
- [17] G. Sidebottom. *A 55-addition rank-23 scheme for 3×3 matrix multiplication*. This repository, 2026. DOI 10.5281/zenodo.21240904.
- [18] G. Sidebottom. *Rigidity of the rank-48 4×4 scheme and The Flower*. Companion notes, this repository, 2026.